COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 212-Automating Server Management	**SECTION:**	CPE31S2 -
DATE PERFORMED:	8/18/26	**DATE SUBMITTED:**	8/18/26
NAME:	Francis Nikko I. Andoy	**INSTRUCTOR:**	Engr. Jimlord Quejado

4.1 Hands-on Activity Ansible Basics

1. PROCEDURE OUTPUT

1. Locally, we use the command sudo apt update when we want to download package information from all configured resources. The sources are often defined in /etc/apt/sources.list file and other files located in /etc/apt/sources.list.d/ directory. So, when you run the update command, it downloads the package information from the Internet. It is useful to get info on an updated version of packages or their dependencies. We can only run an apt update command in a remote machine. Issue the following command:

ansible all -m apt -a update_cache=true

```
[ERROR]: Task failed: Module failed: Failed to lock apt for exclusive operation: Failed to lock directory /var/lib/apt/l
ists/: E:Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)
Origin: <adhoc 'apt' task>

{'action': 'apt', 'args': {'update_cache': 'true'}, 'timeout': 0, 'async_val': 0, 'poll': 15}

ubuntu2 | FAILED! => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.12"
    },
    "changed": false,
    "msg": "Failed to lock apt for exclusive operation: Failed to lock directory /var/lib/apt/lists/: E:Could not open l
ock file /var/lib/apt/lists/lock - open (13: Permission denied)"
}
```

What is the result of the command? Is it successful?

- No, because I don't have the permission to modify the system repository cache files.

Try editing the command and add something that would elevate the privilege. Issue the command ansible all -m apt -a update_cache=true --become --ask-become-pass. Enter the sudo password when prompted. You will notice now that the output of this command is a success. The update_cache=true is the same thing as running sudo apt update. The --become command elevates the privileges and the --ask-become-pass asks for the password. For now, even if we only changed the packaged index, we were able to change something on the remote server. You may notice after the second command was executed, the status is

```
(base) nikko@Nikko:~/CPE212_Andoy-/ansible/hoa4-yarn$ ansible all -i inventory.yml -m apt -a update_cache=true --become
--ask-become-pass
BECOME password:
[WARNING]: Host 'ubuntu2' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation o
f another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-
core/2.19/reference_appendices/interpreter_discovery.html for more information.
ubuntu2 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.12"
    },
    "cache_update_time": 1787034556,
    "cache_updated": true,
    "changed": true
}
```

2. Let's try to install VIM, which is an almost compatible version of the UNIX editor Vi. To do this, we will just changed the module part in 1.1 instruction. Here is the command: ansible all -m apt -a name=vim-nox --become --ask-become-pass. The command would take some time after typing the password because the local machine instructed the remote servers to actually install the package.

```
(base) nikko@Nikko:~/CPE212_Andoy-/ansible/hoa4-yarn$ ansible all -m apt -a name=vim-nox -bK
BECOME password:
[WARNING]: Host 'ubuntu1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation o
f another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-
core/2.19/reference_appendices/interpreter_discovery.html for more information.
ubuntu1 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.12"
    },
    "cache_update_time": 1787034564,
    "cache_updated": false,
    "changed": true,
    "stderr": "",
    "stderr_lines": [],
    "stdout": "Reading package lists...\nBuilding dependency tree...\nReading state information...\nThe following packag
```

- a. Verify that you have installed the package in the remote servers. Issue the command which vim and the command apt search vim-nox respectively. Was the command successful?

```
server-1@Server-1:~$ which vim
/usr/bin/vim
server-1@Server-1:~$
```

- b. Check the logs in the servers using the following commands: cd /var/log. After this, issue the command ls, go to the folder apt and open history.log. Describe what you see in the history.log.

```
Start-Date: 2026-08-18  08:24:12
Commandline: /usr/bin/apt-get -y -o Dpkg::Options::=--force-confdef -o Dpkg::Options::=--force-confold -o DPkg::Lock::Ti
meout=60 install vim-nox=2:9.1.0016-1ubuntu7.18
Requested-By: server-1 (1000)
Install: ruby-sdbm:amd64 (1.0.0-5build4, automatic), zip:amd64 (3.0-13ubuntu0.2, automatic), liblua5.1-0:amd64 (5.1.5-9b
uild2, automatic), fonts-lato:amd64 (2.015-1, automatic), libruby:amd64 (1:3.2~ubuntu1, automatic), ruby-net-telnet:amd6
4 (0.2.0-1, automatic), rubygems-integration:amd64 (1.18, automatic), libruby3.2:amd64 (3.2.3-1ubuntu0.24.04.8, automati
c), rake:amd64 (13.0.6-3, automatic), vim-nox:amd64 (2:9.1.0016-1ubuntu7.18), ruby:amd64 (1:3.2~ubuntu1, automatic), rub
y3.2:amd64 (3.2.3-1ubuntu0.24.04.8, automatic), libjs-jquery:amd64 (3.6.1+dfsg+~3.5.14-1, automatic), unzip:amd64 (6.0-2
8ubuntu4.1, automatic), ruby-rubygems:amd64 (3.4.20-1, automatic), javascript-common:amd64 (11+nmu1, automatic), ruby-xm
lrpc:amd64 (0.3.2-2, automatic), ruby-webrick:amd64 (1.8.1-1ubuntu0.2, automatic)
End-Date: 2026-08-18  08:24:45
server-1@Server-1:~$
```

3. This time, we will install a package called snapd. Snap is pre-installed in Ubuntu system. However, our goal is to create a command that checks for the latest installation package.

- a. Issue the command: ansible all -m apt -a name=snapd --become --ask-become-pass
  Can you describe the result of this command? Is it a success? Did it change anything in the remote servers?

```
(base) nikko@Nikko:~/CPE212_Andoy-/ansible/hoa4-yarn$ ansible all -m apt -a name=snapd -bK
BECOME password:
[WARNING]: Host 'ubuntu1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation o
f another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-
core/2.19/reference_appendices/interpreter_discovery.html for more information.
ubuntu1 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.12"
    },
    "cache_update_time": 1787034564,
    "cache_updated": false,
    "changed": false
}
```

The command is a success it didn't change anything in the remote servers. Because the snapd is already installed in the system so It just returned a success message.

- b. Now, try to issue this command: ansible all -m apt -a "name=snapd state=latest" --become --ask-become-pass

```
(base) nikko@Nikko:~/CPE212_Andoy-/ansible/hoa4-yarn$ ansible all -m apt -a "name=snapd state=latest" -bK
BECOME password:
[WARNING]: Host 'ubuntu2' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation o
f another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-
core/2.19/reference_appendices/interpreter_discovery.html for more information.
ubuntu2 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.12"
    },
    "cache_update_time": 1787034556,
    "cache_updated": false,
    "changed": false
}
```

It returned success because the snapd is at the latest update. Basically what the command did is it asked apt to check the repository for any update. The double quotations is because of the argument we used are 2

Task 2: Writing our First Playbook:

1. With ad hoc commands, we can simplify the administration of remote servers. For example, we can install updates, packages, and applications, etc. However, the real strength of ansible comes from its playbooks. When we write a playbook, we can define the state that we want our servers to be in and the place or commands that ansible will carry out to bring to that state. You can use an editor to create a playbook. Before we proceed, make sure that you are in the directory of the repository that we use in the previous activities (CPE232_yourname). Issue the command nano install_apache.yml. This will create a playbook file called install_apache.yaml. The .yaml is the basic standard extension for playbook files.

2. Run the yml file using the command: ansible-playbook --ask-become-pass install_apache.yml. Describe the result of this command.

```
(base) nikko@Nikko:~/CPE212_Andoy-/ansible/hoa4-yarn$ ansible-playbook -K install_apache.yml
BECOME password:

PLAY [all] *********************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************
[WARNING]: Host 'ubuntu2' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation o
f another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-
core/2.19/reference_appendices/interpreter_discovery.html for more information.
ok: [ubuntu2]
[WARNING]: Host 'ubuntu1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation o
f another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-
core/2.19/reference_appendices/interpreter_discovery.html for more information.
ok: [ubuntu1]

TASK [install apache2 package] *************************************************************************************
ok: [ubuntu2]
ok: [ubuntu1]

PLAY RECAP *********************************************************************************************************
ubuntu1                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu2                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

(base) nikko@Nikko:~/CPE212_Andoy-/ansible/hoa4-yarn$
```

- Basically the playbook executes the tasks which are inside the file then let the user know that it is gathering the needed facts/dependencies for the apache2 to be downloaded.

3. To verify that apache2 was installed automatically in the remote servers, go to the web browsers on each server and type its IP address. You should see something like this.



4. Try to edit the install_apache.yml and change the name of the package to any name that will not be recognized. What is the output?

```
TASK [install apache2 package] ************************************************************************************
[ERROR]: Task failed: Module failed: No package matching 'apache31341' is available
Origin: /home/nikko/CPE212_Andoy-/ansible/hoa4-yarn/install_apache.yml:5:7

3   become: true
4   tasks:
5     - name: install apache2 package
        ^ column 7

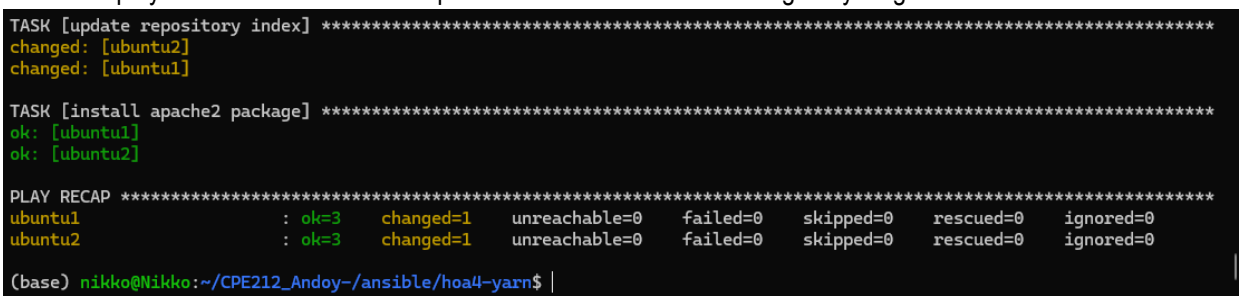
fatal: [ubuntu1]: FAILED! => {"changed": false, "msg": "No package matching 'apache31341' is available"}
fatal: [ubuntu2]: FAILED! => {"changed": false, "msg": "No package matching 'apache31341' is available"}

PLAY RECAP ********************************************************************************************************
ubuntu1                    : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
ubuntu2                    : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
```

- Error because the apt which is the package manager did not able to find the apache313 something in the repository.

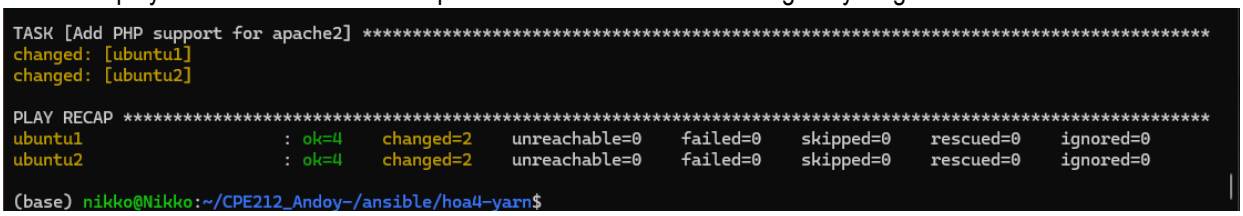
5. This time, we are going to put additional tasks into our playbook. Edit the install_apache.yml. As you can see, we are now adding an additional command, which is the update_cache. This command updates existing package-indexes on a supporting distro but not upgrading installed-packages (utilities) that were being installed.

6. Run the playbook and describe the output. Did the new command change anything on the remote servers?



- Yes there is a change the old metadata is overwritten with the new one because of the update repo index.

7. Edit again the install_apache.yml. This time, we are going to add a PHP support for the apache package we installed earlier.
8. Run the playbook and describe the output. Did the new command change anything on the remote servers?



- In the history.log there is a new installed php which means the command is successful.

9. Finally, make sure that we are in sync with GitHub. Provide the link of your GitHub repository.
https://github.com/illuminado22/CPE212_Andoy-/tree/nikko-v1
Pushed to my own branch (main is empty for now)

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

nswer the following:

1. What is the importance of using a playbook?
   - This is for automation. An example is installation and update using this and integrated with the inventory file for managing the server in just one command. The tasks inside of the yml code is the only thing playbook will run so by this logic preventing unintended duplicate actions on nodes.
2. Summarize what we have done on this activity
   - To summarize this activity we can think of it as automation that can download, update, and service configuration for multiple remote servers as long as the servers are inside of the inventory file.

Activity Summary & Conclusions/Learnings

- In conclusion this activity helped me understand what ansible and yml file is. I also learned how to configure files to make this activity much easier to accomplish. Ansible playbook can help me install and update the software using just the inventory file.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Explanation of concepts and summarization

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.